

Instrucciones de la evaluación ES1

Trabajo individual. Vale el 15 % de tu nota final. Se corrige sobre 30 puntos. Lo construyes durante las 6 sesiones de la unidad.

Cómo se lee este documento

Todo el documento usa siempre las mismas cajas, en el mismo orden, en cada bloque:

- **Idea principal** (caja azul): la frase que hay que entender sí o sí.
- **Puntos clave**: 3 o 4 viñetas cortas.
- **Analogía** (caja verde): una comparación con algo del día a día. Es una comparación para entender, **no** una definición técnica.
- **Palabras nuevas** (caja café clara): las palabras técnicas de **ese mismo bloque**, explicadas ahí mismo. No hay que ir a buscarlas a otra página.

Son 11 bloques. Cada bloque dice en qué número vas, por ejemplo "Bloque 3 de 11".

Esta versión no usa base de datos. Los datos se guardan en un archivo JSON. Las bases de datos se ven en la Unidad 2.

Hay un ejemplo completo y funcionando en GitHub:

github.com/Rixmerz/ejemplo-es1-backend

Resuelve la inscripción a un taller. **Tu problema va a ser otro: no copies el caso, copia la estructura.**

Qué tienes que entregar

Idea principal: Tú eliges el problema que vas a resolver. No hay un enunciado igual para todos.

- Primero escribes un plan corto, apoyándote en una herramienta de IA.
- Después programas ese plan en Python.
- Al final lo muestras en una página web hecha con Django.
- Los datos se guardan en un archivo JSON, no en una base de datos.

Fase 0 · Sesión 1 · Eliges tu problema y escribes el plan	Obligatorio
Fase 1 · Sesiones 1 y 2 · Programa Python que decide	12 puntos
Fase 2 · Sesiones 3 y 4 · Guardas en JSON y muestras con un paquete	6 puntos
Fase 3 · Sesiones 5 y 6 · Lo pasas a Django y escribes ia.md	12 puntos

La Fase 0 no suma puntos, pero **sin el plan la entrega no se recibe**. El plan es lo que permite corregir tu programa contra tu problema y no contra uno genérico.

Dónde va cada archivo. Respeta estas ubicaciones o el proyecto no va a funcionar:

```
miproyecto/  
├─ manage.py  
├─ solucion.py    <- junto a manage.py  
├─ datos.json     <- lo crea solucion.py, aqui mismo  
├─ plan.md, ia.md, requirements.txt  
├─ .env           <- tus claves, NO se sube  
├─ .env.example   <- la plantilla, si se sube  
├─ core/views.py  
├─ core/templates/resumen.html  
└─ miproyecto/settings.py, urls.py
```

PALABRAS NUEVAS DE ESTE BLOQUE

Fase

Una etapa del trabajo. Cada fase se hace en clases distintas.

JSON

Archivo de texto donde se guardan datos ordenados.

Django

Herramienta de Python para hacer páginas web.

Fase 0 · Elige tu problema

Idea principal: Elige algo pequeño y real que hayas visto fallar de verdad. No inventes una empresa.

Ejemplos de problemas del tamaño correcto:

- La lista de espera de un taller del instituto.
- El préstamo de notebooks del laboratorio.
- Los turnos de atención de un policlínico.
- El control de asistencia de un club deportivo.
- El stock de un almacén de barrio.

Tu problema tiene que cumplir estas tres reglas. Si no las cumple, elige otro problema:

- Se decide sobre **una sola cosa a la vez**: una persona, un préstamo, un producto.
- La decisión depende de **al menos dos datos**. Por ejemplo: la edad y los cupos que quedan.
- Existen **4 resultados distintos**: uno de aceptación, dos de rechazo por motivos diferentes, y uno de dato inválido.

La tercera regla no es un capricho. Es lo que obliga a usar **elif** y a combinar condiciones, que es justo lo que se evalúa en el criterio 1.1.2.

PALABRAS NUEVAS DE ESTE BLOQUE

Dato inválido

Un valor que no tiene sentido, como una cantidad de cupos negativa.

Criterio

Cada una de las cuatro cosas que la rúbrica mide para ponerte nota.

Fase 0 · Escribe el plan con ayuda de IA

Idea principal: La IA te ayuda a ordenar el plan. No te escribe el código.

El archivo se llama **plan.md** y tiene dos planas como máximo, con dos apartados:

Apartado de negocio

Problema: qué pasa hoy, a quién le molesta y qué cuesta. **Solución:** qué hace tu programa, en dos frases. **Alcance:** qué entra y qué no entra. **MoSCoW:** la lista de funciones priorizada. Se explica completa en el bloque 4.

Apartado técnico

Datos de entrada: qué le pides al usuario y de qué tipo es cada dato. **Regla de decisión:** los 4 resultados y cuándo ocurre cada uno. **Paquete externo:** cuál vas a usar y para qué. **Pantalla web:** qué dirección tendrá y qué muestra.

Escríbele esto a la herramienta de IA y después corrige lo que no calce con tu problema:

Soy estudiante de programacion back end, primer semestre.
Quiero resolver este problema: [ESCRIBE TU PROBLEMA EN 3 LINEAS].

Ayudame a escribir un plan de 2 planas con dos apartados:

- 1) NEGOCIO: problema, solucion, alcance y priorizacion MoSCoW (Must, Should, Could, Won't).
- 2) TECNICO: datos de entrada con su tipo, la regla de decision con 4 resultados distintos, y que mostraria una sola pantalla web.

Restriccion: se resuelve con variables, if/elif, un archivo JSON y una sola vista Django. Sin base de datos, sin login, sin API.

La última línea es la parte importante. Si no se la escribes, la IA te va a proponer base de datos y login, y vas a planificar algo que no alcanzas a construir. Guarda esta conversación: la vas a necesitar para **ia.md**.

PALABRAS NUEVAS DE ESTE BLOQUE

Alcance

El límite de lo que tu programa hace y de lo que no hace.

Paquete externo

Código hecho por otras personas que instalas para no escribirlo tú.

Vista

En Django, la función que arma lo que se ve en una página.

MoSCoW · decide qué vas a programar

Idea principal: Escribes todas las funciones que se te ocurran, las clasificas en cuatro grupos, y programas solo el primer grupo.

MoSCoW ordena las funciones por prioridad. El nombre viene de las cuatro categorías:

Must · imprescindible

Sin esto el programa no sirve. **Es lo único que programas.** Deja 3 a 5 funciones acá.

Ejemplo: pedir los datos, decidir, mostrar el motivo, guardar en JSON.

Should · importante

Suma bastante, pero el programa funciona sin ello. Queda escrito y no se programa.

Ejemplo: corregir un registro mal ingresado.

Could · deseable

Estaría bueno si sobrara tiempo. Nunca sobra. **Ejemplo:** exportar el resumen a Excel.

Won't · fuera por ahora

Decides a propósito que no va en esta versión. **Ejemplo:** cuentas de usuario con contraseña.

Al grupo de los **Must** se le llama **MVP**. Ese es tu compromiso: es contra esa lista que se corrige tu programa.

ANALOGÍA

Es como armar la maleta para un viaje de tres días. El pasaporte es Must. Un segundo par de zapatos es Should. La cámara es Could. La plancha de ropa es Won't: cabe, pero decides que no va.

PALABRAS NUEVAS DE ESTE BLOQUE

MoSCoW

Forma de ordenar funciones en cuatro grupos según qué tan necesarias son.

MVP

La versión más pequeña que ya sirve para algo. Son tus Must.

Función

Acá significa "una cosa que el programa hace", no la instrucción de Python.

Fase 1 · El programa en Python · 12 puntos

Idea principal: Un programa de consola que pide datos y responde con uno de los 4 resultados de tu plan.

Sesiones 1 y 2. El archivo se llama **solucion.py**. Haz esto en orden:

- **Paso 1.** Declara una variable por cada dato de entrada de tu plan.
- **Paso 2.** Lo que entrega **input()** siempre es texto. Si es un número, conviértelo con **int()**.
- **Paso 3.** Escribe la regla de decisión con **if**, **elif** y **else**, cubriendo los 4 resultados.
- **Paso 4.** Cada resultado muestra un mensaje distinto que dice el motivo.
- **Paso 5.** Prueba los 4 casos con datos reales antes de seguir.

El orden de las condiciones importa: pon **primero** el caso del dato inválido. Si lo dejas al final, nunca se alcanza a revisar.

```
nombre = input("Nombre: ")
edad = int(input("Edad: "))
cupos = 3

if cupos < 0:
    print("Error: cantidad de cupos invalida")
elif edad >= 16 and cupos > 0:
    print(f"{nombre}: aceptado")
elif ... # completa los otros dos rechazos
```

ANALOGÍA

El programa funciona como el guardia en la puerta de un evento. Mira dos cosas, la edad y si queda espacio, y según eso te deja pasar o te dice el motivo exacto por el que no.

Es una comparación para entender, no una definición técnica.

PALABRAS NUEVAS DE ESTE BLOQUE

Variable

Un nombre que guarda un dato para usarlo después.

input()

Instrucción que le pide un dato a la persona que usa el programa.

int()

Instrucción que convierte un texto en número entero.

elif

"Si no se cumplió lo anterior, revisa esta otra condición".

Fase 2 · Guardar en JSON y mostrar · 6 puntos

Idea principal: Cada decisión se guarda en un archivo JSON y al final se muestra un resumen con formato de tabla.

Sesiones 3 y 4. Son dos cosas distintas y las dos son necesarias:

- **Guardar:** se hace con **json**, que ya viene con Python. No se instala.
- **Mostrar:** se hace con **tabulate**, que sí hay que instalar con pip. Esto es lo que da los 6 puntos.

```
pip install tabulate
pip freeze > requirements.txt
```

```
import json
from tabulate import tabulate

registros.append({"nombre": nombre, "estado": "Aceptado"})

with open("datos.json", "w") as f:
    json.dump(registros, f, indent=2)

print(tabulate(registros, headers="keys"))
```

Instalar el paquete e importarlo **no basta**. La rúbrica pide que el paquete se use de verdad en tu solución y que la tabla salga bien formada.

ANALOGÍA

El archivo JSON es como el cuaderno donde el guardia anota a cada persona que pasó. Se queda escrito aunque el programa se cierre. Tabulate es pasar esas anotaciones en limpio a una tabla ordenada.

Es una comparación para entender, no una definición técnica.

PALABRAS NUEVAS DE ESTE BLOQUE

pip

El instalador de paquetes de Python.

requirements.txt

La lista de paquetes que tu proyecto necesita para funcionar.

json.dump()

Instrucción que escribe tus datos dentro del archivo JSON.

Fase 3 · Django · 12 puntos

Idea principal: Una sola pantalla web que muestra el resultado. Sin base de datos: la vista lee el archivo JSON.

Sesiones 5 y 6. Es el criterio que más pesa. En orden:

```
pip install django
django-admin startproject miproyecto
cd miproyecto
python manage.py startapp core
```

- **Paso 1.** Registra la app **core** en **INSTALLED_APPS**, dentro de settings.py. No es un trámite: si no lo haces, Django **no encuentra tu template** y falla con **TemplateDoesNotExist**.
- **Paso 2.** Escribe una vista que **reutilice** tu regla de decisión de la Fase 1. No la escribas de nuevo: impórtala.
- **Paso 3.** La vista abre **datos.json** y le pasa el contenido al template.
- **Paso 4.** Conecta la vista a una dirección en **urls.py**.
- **Paso 5.** Crea el template en **core/templates/resumen.html**. Django solo lo encuentra en esa carpeta.
- **Paso 6.** Levanta con **python manage.py runserver** y revisa tu dirección.
- **Paso 7.** No crees modelos ni ejecutes migraciones: no hay base de datos.

ANALOGÍA

La vista es el mesero: va a la cocina, que es tu JSON, y trae lo que hay a la mesa, que es la pantalla.

PALABRAS NUEVAS DE ESTE BLOQUE

Template

El archivo HTML donde Django pone tus datos para mostrarlos.

urls.py

Conecta cada dirección web con su vista.

runserver

Comando que enciende el servidor de prueba.

Migración

Paso que crea tablas en una base de datos. Aquí no se usa.

Fase 3 · Las claves no van en el código

Idea principal: La clave secreta de Django se guarda en un archivo aparte llamado `.env`, que nunca se sube a internet.

Cuando creas el proyecto, Django escribe una **SECRET_KEY** dentro de `settings.py`. Si subes ese archivo a GitHub, tu clave queda pública. Se saca así:

```
pip install python-decouple
```

```
# settings.py
from decouple import config

SECRET_KEY = config('SECRET_KEY')
DEBUG = config('DEBUG', default=False, cast=bool)
```

```
# .env (este archivo NO se sube)
SECRET_KEY=clave-de-ejemplo-solo-para-clases
DEBUG=True
```

- Crea también **.env.example**, igual pero sin los valores reales. Ese **sí** se sube: le dice a quien reciba tu proyecto qué variables necesita.
- Agrega **.env** a tu archivo **.gitignore** para que git lo ignore.
- Si falta el `.env`, Django corta con **UndefinedValueError**. Es a propósito: es mejor que falle a que arranque con una clave equivocada.

ANALOGÍA

El `.env` es la llave de tu casa y el `.env.example` es la foto de la cerradura. Puedes mostrar la foto para que sepan qué llave hace falta, sin entregar la llave.

PALABRAS NUEVAS DE ESTE BLOQUE

SECRET_KEY

Clave que Django usa para firmar. Es privada.

.env

Archivo con las claves. Nunca se sube a internet.

.gitignore

Lista de archivos que git debe ignorar.

Fase 3 · La vista que lee el JSON

Idea principal: La vista abre el archivo, y si todavía no existe muestra la página vacía en vez de caerse.

```
# core/views.py
import json, os
from django.shortcuts import render

def resumen(request):
    registros = []
    if os.path.exists("datos.json"):
        with open("datos.json") as f:
            registros = json.load(f)
    return render(request, "resumen.html",
                  {"registros": registros})
```

- Sin las dos líneas de **import** el programa falla con **NameError**. Van siempre arriba.
- El **if os.path.exists** evita el error cuando datos.json todavía no existe, que es lo que pasa la primera vez que abres la página.
- Prueba las dos situaciones: con datos.json y sin él. Las dos tienen que mostrar la página.

En el ejemplo de GitHub esta vista está resuelta y funcionando, junto con el template que muestra la tabla.

PALABRAS NUEVAS DE ESTE BLOQUE

import

Línea que trae código de otro lado para poder usarlo.

NameError

Error que aparece cuando usas un nombre que Python no conoce.

os.path.exists

Pregunta si un archivo existe. Responde sí o no.

El archivo **ia.md** y la entrega

Idea principal: Usar IA está permitido y es parte del trabajo. Lo que se evalúa es que entiendas lo que entregas.

El archivo **ia.md** es media plana, en tus palabras, con tres cosas:

- Qué herramienta de IA usaste y para qué la consultaste.
- Una consulta concreta que hiciste y qué te respondió.
- Qué de esa respuesta estaba mal, sobraba o no entendiste, y cómo lo corregiste tú.

Sin **ia.md** el criterio 1.1.4 no llega a Logrado, aunque el proyecto funcione perfecto. Y si no puedes explicar una línea de tu código en clase, esa parte no se considera lograda.

plan.md · tu problema, apartado de negocio y técnico	Requisito
solucion.py · el programa de consola	1.1.1 · 1.1.2 · 1.1.3
datos.json · los registros guardados	1.1.3
requirements.txt · el paquete instalado con pip	1.1.3
.env.example · la plantilla de las claves (el .env NO se sube)	1.1.4
miproyecto/ · el proyecto Django	1.1.4
ia.md · qué consultaste y qué corregiste	1.1.4

Todo en una carpeta comprimida llamada **ES1_ApellidoNombre.zip**, subida al AAI.

PALABRAS NUEVAS DE ESTE BLOQUE

Logrado

El nivel más alto de la rúbrica, el que da todos los puntos del criterio.

AAI

La plataforma del instituto donde subes la entrega.

El ejemplo de referencia

Idea principal: Hay un proyecto completo y funcionando en GitHub. Muestra la estructura que se espera de tu entrega, resuelta de punta a punta.

github.com/Rixmerz/ejemplo-es1-backend

De qué trata. Resuelve la inscripción a un taller de robótica con 3 cupos. Pide nombre y edad, decide si la persona queda inscrita, guarda el registro en datos.json y lo muestra en una página web con un formulario. Un rechazado queda registrado pero no ocupa cupo.

Cómo cumple cada criterio. Esta tabla te sirve de checklist contra tu propio proyecto:

1.1.1 · nombre es texto, edad pasa por int(), y los cupos se calculan restando	solucion.py
1.1.2 · los 4 resultados con comparaciones y un and	decidir()
1.1.3 · tabulate instalado con pip, importado y usado	solucion.py
1.1.4 · levanta, la URL resuelve, la vista responde, el template muestra	core/ + ia.md

La vista **no reescribe** la regla: la importa desde solucion.py. Así la lógica vive en un solo lugar.

ANALOGÍA

Es como la maqueta que el profesor deja en la sala: la miras para entender cómo se arma, pero la tuya tiene que ser de tu propio tema.

No copies el caso del taller. Tu problema es otro. Lo que se copia es la estructura: qué archivo va dónde y qué hace cada uno.

PALABRAS NUEVAS DE ESTE BLOQUE

Repositorio

Carpeta de un proyecto guardada en GitHub.

Clonar

Bajar a tu computador una copia del repositorio.

Resumen en 7 líneas

- Eliges tú un problema pequeño y real, con 4 resultados posibles.
- Escribes plan.md con IA: apartado de negocio y apartado técnico.
- Priorizas con MoSCoW y programas solo los Must, que son tu MVP.
- Programas la regla de decisión en solucion.py con if y elif.
- Guardas los registros en datos.json y los muestras con tabulate.
- Pasas el resultado a una pantalla web en Django, sin base de datos.
- Sacas la clave secreta del código y la dejas en un archivo .env.
- Escribes ia.md y entregas todo en un zip con tu apellido y nombre.

Cómo se te pone la nota

1.1.1 · Variables y operaciones · solucion.py	6 puntos
1.1.2 · Instrucciones, estructuras y operadores · solucion.py	6 puntos
1.1.3 · Paquete externo · solucion.py y requirements.txt	6 puntos
1.1.4 · Aplicación Django · miproyecto/ e ia.md	12 puntos
Total	30 puntos

La rúbrica completa, con los cuatro niveles de cada criterio, está en el AAI. Léela antes de entregar.

Actividad de hoy · 15 minutos

Completa esta ficha ahora, en clases, y muéstrala antes de salir. Es el punto de partida de tu plan.md.

1. ¿Qué problema real vas a resolver?

2. ¿Qué dos datos le vas a pedir al usuario?

3. ¿Cuál es el caso en que la respuesta es "aceptado"?

4. ¿Cuáles son los dos motivos distintos de rechazo?

5. ¿Qué dato sería inválido en tu problema?

Antes de entregar, revisa

- plan.md tiene los dos apartados y programé exactamente mis Must.
- Mi programa corre sin errores desde una terminal recién abierta.
- Probé los 4 resultados y cada uno da un mensaje distinto.
- datos.json se crea y guarda los registros.
- requirements.txt incluye el paquete externo.
- El servidor Django levanta y mi dirección responde.
- La SECRET_KEY está en .env, el .env está en .gitignore y subo .env.example.
- ia.md está escrito y dice qué corregí yo.
- El archivo se llama ES1_ApellidoNombre.zip.

TI3V41 · Programación Back End · Unidad 1 · Docente: Juan Pablo Díaz S.